

수업 08 · 2026-10-23

전자회로·ESP32

전압, 전류, 저항을 배우고 ESP32의 입출력을 살펴봅니다. 회로를 종이로 검토하고 가상 센서 기록을 읽어 봅니다.

10:00~17:00 · 50분 수업 후 10분 휴식 · 중식 11:50~13:00

오늘 필수 활동 10개 · 교시별 설명과 예제를 보고 직접 해 봅니다.

2026-10-23 · 처음 시작

이번 수업에서는 실제 연결·통전·시리얼 포트 열기를 하지 않습니다. Python 표준 라이브러리로 가상 센서 파일을 읽습니다. .ino는 모델 확정 후 실행할 완성 예의 읽기 자료이며, 현재 컴파일·업로드 결과가 아닙니다.

파일과 실행

examples/course/data는 읽을 입력, code는 완성 시연 코드, work는 나의 작성·수정 파일입니다. ZIP을 풀고 폴더 구조를 유지합니다. 터미널은 명령을 실행하는 창이며, 에디터는 파일 내용을 쓰고 저장하는 창입니다. 저장만으로 코드가 실행되지는 않습니다.

1. 용어에서 낯선 말을 확인합니다.
2. 교안의 개념과 예를 읽습니다.
3. 활동지의 A01부터 입력을 열고 work 양식에 기록합니다.
4. 코드 실행은 code/line-guide.md의 명령과 조건을 따릅니다.

수업을 마칠 때 확인할 것: 전압·전류·저항의 단위를 구분하고 0·null·문자열을 다른 상태로 처리하며 예제로 확인한 것과 장비에서 더 시험할 것을 설명합니다.

미확인 환경: 정확한 보드·센서·저항·LED 모델과 수량은 미확인이다. 전원·핀·배선을 추측하여 통전하지 않는다. 개인 코드 실행은 Python3를 사용할 수 있는 PC가 조건이다. 중식은 11:50~13:00이다. 학생 수행시간은 수업 후 기록한다.

수업 전, 이 세 가지부터 확인하세요

앞선 내용이 기억나지 않으면 아래 질문부터 읽어 보세요. 먼저 답을 적고 풀이와 비교해 보세요.

단위

질문: 3.3V와 3.3mA를 같은 크기로 비교하나요?

풀이: 전압은 밀어주는 정도, 전류는 흐르는 양이며 다른 물리량입니다. 숫자만 같다고 같은 조건이 아닙니다.

회로 경로

질문: LED와 저항을 놓기만 하면 전류가 흐르나요?

풀이: 전원에서 부품을 거쳐 돌아오는 연결 경로가 필요합니다. 회로 검토와 실제 전원 인가를 구분합니다.

판정

질문: 모의 입력에서 명령이 나왔다면 실물 LED도 켜졌나요?

풀이: 코드의 판단 결과를 확인한 것입니다. 핀 연결·전압·보드 환경을 확인한 실물 검증은 별도입니다.

오늘의 수업과 복습을 나누는 방법

수업은 10:00~10:50, 11:00~11:50, 13:00~13:50, 14:00~14:50, 15:00~15:50, 16:00~16:50입니다. 중식은 11:50~13:00이며 수업 사이에는 10분 쉽니다. 마지막 16:50~17:00은 휴식과 저장·퇴실 준비로 배치했습니다.

교시	수업에서 먼저 확인할 활동	추가·복습
1	A01, A02, A03	새 입력으로 한 번 더 확인
2	A04, A05, A06	새 입력으로 한 번 더 확인
3	A07, A08, A09	새 입력으로 한 번 더 확인
4	A10, A11, A12	새 입력으로 한 번 더 확인
5	A13, A14, A15	새 입력으로 한 번 더 확인
6	A16, A17, A18	새 입력으로 한 번 더 확인

교안의 쉬운 사례와 단계별 풀이를 먼저 읽고 필수 결과를 직접 만들어 봅니다. 새 문제는 결과를 확인한 뒤 복습하거나 도전합니다. 모든 페이지를 같은 시간에 끝내라는 뜻은 아닙니다. 설치가 막혔을 때는 제공 출력과 수동 추적으로 개념을 확인하고, 직접 실행했는지는 따로 적습니다.

값을 읽는 코드가 실제 세상과 만나려면 전기 신호의 약속이 필요합니다. 오늘은 회로를 종이에서 읽고, 센서 대신 제공 파일을 실행하여 신호·값·판단의 관계를 확인합니다. 실제 연결과 통전은 하지 않습니다.

수업의 출발점

물리량의 단위→회로의 경로와 기준→보드와 신호→디지털 기록→검수 순서로 학습합니다. 하드웨어 모델이 정해지지 않았으므로 모든 구체 핀 번호와 배선은 보류하며, 모의 값으로 확인할 수 있는 범위만 실제 실행합니다.

읽는 방법

먼저 개념을 읽고 예제의 입력값과 예상 결과를 살펴봅니다. 수업은 50분 × 6교시로 총 300분이며 중식은11:50~13:00입니다. 각 교시에 개념 설명·시연·직접 실습·풀이가 있습니다. 복습·도전 문제는 기본 실습을 마친 뒤 선택하며, 시간이 부족하면 수업 뒤에 이어서 풀입니다. 학생 실습에 걸리는 시간은 수업 후 기록합니다.

1교시 · 전기 기초와 계산

10:00~10:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A01 · 전압·전류·저항을 구분

1교시 필수 개념 보충 · 전류가 돌아오는 길과 단위 계산

실무 상황과 원리

실습 담당자가 3.3V 전원과 330Ω 저항을 종이에 연결합니다. “3.3이 흐른다”라는 말로는 무엇을 측정하는지 알 수 없습니다. 전압은 두 점 사이의 차이이고 단위는 V입니다. 전류는 시간당 이동하는 전하량이며 A, 저항은 전압과 전류의 관계를 나타내며 Ω입니다. 물의 압력 차이와 유량 비유는 역할을 기억하는 출발점으로부터만 사용합니다.

전류의 통상적인 방향을 전원 + → 저항 → 전원 -로 표시합니다. 돌아오는 길까지 이어져야 닫힌 회로입니다. 중간이 끊어진 이상적인 회로에는 지속적인 전류가 흐르지 않습니다. GND는 이 회로에서 전압 비교의 기준으로 정한 지점이며 언제나 건물의 보호 접지라는 뜻은 아닙니다. 저항 없이 전원 +와 -를 잇는 것은 부하를 통과하는 정상 경로와 다릅니다.

완성 계산의 입력은 저항 양단 전압 3.3V와 저항 330Ω입니다. 옴성 저항의 관계 $V=IR$ 에서 $I=V/R$ 로 바꿉니다. $3.3 \div 330 = 0.010A$ 이고, $1A=1000mA$ 이므로 10mA입니다. 저항을 660Ω으로 바꾸면 $0.005A=5mA$ 입니다. 나눗셈 결과에 바로 mA를 붙이면 1000배의 단위 오류가 됩니다. 계산은 이상적인 저항의 관계를 확인한 것이며 부품 발열, 허용 전류, 보드 출력 능력을 승인한 결과는 아닙니다. 실제 연결 전에는 부품과 보드 사양이 필요합니다. 모의 계산의 실행 여부와 실물 측정 여부를 기록 칸에서 나눕니다.

새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: 저항 양단 5V와 1000Ω이라는 새 종이 조건에서 A와 mA를 각각 계산하고 전류가 돌아오는 경로를 화살표로 그리세요. 가운데 선을 지웠을 때의 이상적 전류도 쓰세요.

정답과 오답의 이유

$0.005A=5mA$ 이며 + → 저항 → -입니다. 선이 끊어진 이상적 경로는 0A입니다. 5V를 ESP32 신호 입력에 연결해도 된다는 뜻은 아닙니다. 단위 계산과 연결 허용 판단은 별도 문제입니다.

확인 질문과 답

1. 전압은 몇 점 사이인가요? → 두 점 사이입니다.
2. 0.002A는? → 2mA입니다.
3. GND는 버리는 구멍인가요? → 전압 기준과 전류가 돌아오는 길의 역할을 읽어야 합니다.

전압은 두 점 사이의 전기적 에너지 차이를 전하량으로 나눈 값이고 단위는 V입니다. 전류는 시간당 흐르는 전하량이며 A, 저항은 전류 흐름을 제한하는 관계를 나타내며 Ω로 씁니다.

단위가 다른 수를 같은 값처럼 연결하면 회로 계산과 부품 선택을 잘못합니다. 수도관의 압력차·유량·흐름 저항 비유는 역할을 기억하는 데만 씁니다.

입력에서 살펴볼 예

전압3.3V 전류0.010A=10mA 저항330Ω

전압은 두 점 사이에서 말합니다. 전류는 흐름이며, 저항은 그 흐름과 전압의 관계입니다.

구분해야 할 반례

전압 자체가 회로를 흐른다고 말하지 않습니다. 물 비유로 전기 현상의 모든 성질을 설명할 수 없습니다.

스스로 설명할 질문: 3.3V와10mA를 서로 바꿔 쓸 수 없는 이유는 무엇인가요?

활동 연결: [입력](#) · [기록 양식](#)

A01 · 직접 해보기

A01 · 전압·전류·저항을 구분

목적 단위가 다른 수를 같은 값처럼 연결하면 회로 계산과 부품 선택을 잘못합니다. 수도관의 압력차·유량·흐름 저항 비유는 역할을 기억하는 데만 씁니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A01_input.md](#)

작업: [A01_worksheet.md](#)

단계

1. 세 값을 이름과 단위에 연결합니다.
2. A를mA로 바꿉니다.
3. 전압이 어느 두 점 사이인지 질문을 작성합니다.

예상 결과 입력 근거를 표시하고 물리량과 단위에 판단을 남깁니다.

확인 방법 3.3V와10mA를 서로 바꿔 쓸 수 없는 이유는 무엇인가요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

A03 · 옴의 법칙을 계산으로 확인

일정 조건에서 옴성 저항은 $V=IR$ 관계를 따릅니다. 전류를 구할 때 $I=V/R$ 로 바꾸고 단위를 맞춥니다. LED 같은 비선형 부품 전체에 고정 저항처럼 적용하지 않습니다.

전압이 같을 때 저항을 두 배로 하면 이상적인 계산 전류가 절반이라는 관계를 확인합니다. 계산 결과와 부품의 실제 허용 정격은 별개입니다.

입력에서 살펴볼 예

저항 양단3.3V, 저항330Ω $I=3.3/330=0.010A$ 660Ω로 변경

330Ω 계산은10mA입니다. 660Ω 계산은5mA입니다. 부품 정격과 발열은 따로 확인합니다.

구분해야 할 반례

이 계산만으로 ESP32핀에서10mA를 흘려도 된다고 승인할 수 없습니다. 보드·칩·핀·부품의 문서가 더 필요합니다.

스스로 설명할 질문: 계산 전류와 실제 연결 허용 여부는 왜 다른가요?

활동 연결: [입력](#) · [기록 양식](#)

01_ohm.py 전체

```

voltage_v = 3.3
resistance_ohm = 330
if resistance_ohm <= 0:
    raise ValueError("저항은 양수여야 합니다")
current_a = voltage_v / resistance_ohm
current_ma = current_a * 1000
print("current_mA:", round(current_ma, 2))
    
```

실행 조건: Python 3 표준 라이브러리만 사용합니다. 장치·포트·전원을 연결하지 않습니다.

실행 폴더: examples/course

명령: `python -X utf8 code/01_ohm.py`

결과:

```

current_mA: 10.0
    
```

각 줄의 뜻은 [코드 해설](#)에 있습니다.

작은 수정: 저항을 660으로 바꾸면 5.0mA입니다. 전압은 3.3V를 유지합니다.

오류 복구: 저항0은 단락 모사 결과가 아니라 입력 오류입니다. 실제 단락을 만들어 확인하지 않습니다.

A03 · 직접 해보기

A03 · 옴의 법칙을 계산으로 확인

목적 전압이 같을 때 저항을 두 배로 하면 이상적인 계산 전류가 절반이라는 관계를 확인합니다. 계산 결과와 부품의 실제 허용 정격은 별개입니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A03_input.md](#) · [01_ohm.py](#)

작업: [A03_worksheet.md](#)

단계

1. 코드 각 줄을 읽고 결과를 예상합니다.
2. 원본과 660Ω 수정본을 비교합니다.
3. 0Ω 입력을 실제 연결 대신 코드 오류로 확인합니다.

예상 결과 입력 근거를 표시하고 계산 비교에 판단을 남깁니다.

확인 방법 계산 전류와 실제 연결 허용 여부는 왜 다른가요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 닫힌 경로와 GND

[이 교시의 다른 예와 코드 해설 열기](#)

2교시 · 부품·보드·입출력

11:00~11:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A04 · LED와 저항의 역할

2교시 필수 개념 보충 · LED 직렬 저항과 보드 신호의 전압

실무 상황과 원리

알림등을 달고 싶은 담당자는 LED가 밝아지면 성공이라고 생각합니다. 그러나 빛이 보이는 것과 부품이 정격 안에서 동작하는 것은 다릅니다. LED는 방향이 있는 다이오드이며 저항처럼 일정한 V/I 값으로 취급하지 않습니다. 직렬 저항은 같은 가지를 흐르는 전류를 제한합니다.

저항에 걸리는 전압은 $3.3 - 2.0 = 1.3\text{V}$ 입니다. 전류는 $1.3 \div 330 = 0.00394\text{A}$, 약 3.94mA 입니다. 저항 양단에 3.3V 전체가 걸린다고 계산한 10mA 와 다릅니다. 목표 전류를 가상으로 5mA 라 정하면 필요한 이상적 저항은 $1.3 \div 0.005 = 260\Omega$ 입니다. 실제 선택은 LED 사양, 허용 전류, 저항 정격, 오차와 사용 가능한 표준값을 확인하는 후속 작업입니다. 오늘 숫자는 배선 승인값이 아닙니다.

칩은 계산과 입출력을 수행하는 핵심 부품, 모듈은 칩과 주변 회로 묶음, 개발 보드는 연결과 개발을 돕는 전체 기판입니다. 보드에 5V 전원 단자가 있어도 GPIO 입력이 5V를 허용한다는 결론은 나오지 않습니다. GPIO는 프로그램으로 디지털 신호를 읽거나 출력하는 핀이며 HIGH는 정해진 논리 수준을 뜻합니다. 핀 번호, 부팅 관련 기능, 출력 전류 제한은 정확한 ESP32 계열과 보드 문서로 확인해야 합니다. 모델이 미확정이면 구체 배선 대신 확인할 항목을 작성합니다.

새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: 공급 3.3V, 가정 LED 강하 1.8V, 저항 500Ω에서 전류를 계산하세요. 보드 전원 입력 표시에 5V가 있다는 이유로 센서의 5V 출력을 GPIO에 연결할 수 있는지도 판단하세요.

정답과 오답의 이유

저항 전압은 1.5V, 전류는 3mA입니다. GPIO 허용 전압은 별도 사양으로 확인해야 합니다. LED 강하를 더해 5.1V로 계산하거나 USB 전원과 신호 입력 정격을 같은 것으로 읽으면 오답입니다.

확인 질문과 답

1. 직렬 가지의 전류는? → 같은 가지에서 같습니다.
2. 2.0V는 확정 사양인가요? → 여기서는 계산용 가정입니다.
3. 모델 미확정 시 핀 번호는? → 확정하지 않고 확인 항목으로 남깁니다.

짧은 완성 코드로 추적하기

제공 참고 파일 [code/05_led_calculation.py](#)은 보존합니다. 예제 [course](#) 폴더에서 이 파일을 [work/05_led_calculation.py](#)에 복사하고 학생 수정본은 work에 저장합니다. 아래는 완결된 전체 코드입니다. 파일·폴더·확장자를 확인한 뒤 같은 [course](#) 폴더의 터미널에서 `python -X utf8 work/05_led_calculation.py`을 실행합니다. 저장과 실행은 별도 행동입니다.

```
supply_v = 3.3
led_v = 2.0
resistance_ohm = 330

if resistance_ohm <= 0 or supply_v < led_v:
    raise ValueError("계산 가정을 확인하세요")

current_ma = (supply_v - led_v) / resistance_ohm * 1000
print("assumed_LED_mA:", round(current_ma, 2))
```

예상 출력:

```
assumed_LED_mA: 3.94
```

1~3행은 공급 전압·가정 LED 강하·저항을 이름에 저장합니다. =는 오른쪽 값을 왼쪽 이름에 붙이는 기호입니다. 4행의 <=는 작거나 같음, <는 작음, or는 둘 중 하나라도 참이라는 뜻입니다. 콜론 뒤 들여쓴 5행은 잘못된 가정을 오류로 알립니다. 6행의 괄호는 뺄셈을 먼저 하게 하며 /는 나눗셈, *는 곱셈입니다. 1000은 A를 mA로 바꾸는 수입니다. 7행은 round로 소수 둘째 자리까지 반올림해 출력합니다. 저항을 660으로 바꾸면 1.97입니다. 저항 0은 오류이므로 330으로 복구합니다. 계산 실행에는 보드와 전원을 연결하지 않습니다.

LED는 전류가 흐를 때 빛을 내는 다이오드이며 방향과 동작 조건이 있습니다. 직렬 저항은 회로 전류를 제한하는 데 사용됩니다. LED의 전압 강하는 부품과 전류에 따라 달라집니다.

같은 330Ω이라도 저항 양단 전압이 달라지면 전류가 달라집니다. LED가 있는 회로를 저항 하나의 계산과 구분합니다.

입력에서 살펴볼 예

종이 가정: 공급3.3V LED 전압 강하2.0V 저항330Ω $I = (3.3 - 2.0) / 330$

가정하에서 약3.94mA입니다. 2.0V는 이번 계산용 가정입니다. 실물 LED 사양은 미확인입니다.

구분해야 할 반례

저항 없이 LED를 직접 연결하라는 지시가 아닙니다. LED모델·정격·극성·핀 허용 조건을 확인하기 전에는 통전하지 않습니다.

스스로 설명할 질문: A03의10mA와 여기3.94mA가 다른 이유는 무엇인가요?

활동 연결: [입력](#) · [기록 양식](#)

A04 · 직접 해보기

A04 · LED와 저항의 역할

목적 같은330Ω이라도 저항 양단 전압이 달라지면 전류가 달라집니다. LED가 있는 회로를 저항 하나의 계산과 구분합니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A04_input.md](#)

작업: [A04_worksheet.md](#)

단계

1. 저항에 걸리는 전압1.3V를 구합니다.
2. 전류를 mA로 계산합니다.
3. 실물 전에 확인할 문서 항목 세 개를 적습니다.

예상 결과 입력 근거를 표시하고 LED 계산에 판단을 남깁니다.

확인 방법 A03의10mA와 여기3.94mA가 다른 이유는 무엇인가요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

A06 · GPIO와 논리 수준

GPIO는 프로그램으로 입출력 역할을 정하는 범용 핀입니다. 디지털 입력은 정해진 전압 범위를 LOW/HIGH로 해석합니다. 논리값은 물리 전압의 약속과 연결되지만 같은 단위는 아닙니다.

입력·출력 방향을 혼동하거나 전압 범위를 넘기면 코드가 맞아도 장치 문제가 생깁니다. GPIO와 공급 핀을 먼저 구분합니다.

입력에서 살펴볼 예

신호 계획 버튼 상태 → 디지털 입력 표시 상태 → 디지털 출력 구체 GPIO 번호 → 미정

pinMode는 방향을 정하는 API입니다. digitalRead는 논리 상태를 읽습니다. 번호·허용 전압은 모델 문서로 확인합니다.

구분해야 할 반례

HIGH가 모든 보드에서 5V라는 뜻은 아닙니다. USB에 5V가 들어간다고 GPIO가 5V 입력을 허용한다고 추정하지 않습니다.

스스로 설명할 질문: 전원 입력과 신호 입력을 왜 구분해야 하나요?

활동 연결: [입력](#) · [기록 양식](#)

A06 · 직접 해보기

A06 · GPIO와 논리 수준

목적 입력·출력 방향을 혼동하거나 전압 범위를 넘기면 코드가 맞아도 장치 문제가 생깁니다. GPIO와 공급 핀을 먼저 구분합니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A06_input.md](#)

작업: [A06_worksheet.md](#)

단계

1. 신호마다 입력·출력을 표시합니다.
2. pinMode/digitalRead 역할을 연결합니다.
3. 전압·핀맵 확인 전 보류 결정을 적습니다.

예상 결과 입력 근거를 표시하고 입출력 계획에 판단을 남깁니다.

확인 방법 전원 입력과 신호 입력을 왜 구분해야 하나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 칩·모듈·개발 보드

[이 교사의 다른 예와 코드 해설 열기](#)

3교시 · 변환·스키마·모의입력

13:00~13:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A09 · 모의 센서 파일을 읽기

모의 입력은 실제 장치 대신 조건을 통제된 값입니다. 프로그램의 분기와 출력은 시험할 수 있지만 센서의 오차나 연결 품질은 시험하지 못합니다.

같은 입력으로 조건 변경 전후를 비교할 수 있어 실물 연결 전 처리 논리를 확인하기 좋습니다.

입력에서 살펴볼 예

02_sensor_replay.py 기준 28.0°C 이상 alert 숫자 이외는 invalid null은 missing

1 normal · 2 alert 3 missing · 4 normal 5 invalid

구분해야 할 반례

출력 alert는 팬이나 릴레이를 작동시킨 결과가 아닙니다. 화면용 상태 이름이며 실제 제어 명령이 없습니다.

스스로 설명할 질문: 모의 입력으로 확인한 것과 못 한 것은 무엇인가요?

활동 연결: [입력](#) · [기록 양식](#)

02_sensor_replay.py 전체

```
import json
from pathlib import Path

root = Path(__file__).resolve().parent.parent
source = root / "data" / "sensor_samples.json"

samples = json.loads(source.read_text(encoding="utf-8"))

for sample in samples:
    value = sample["temperature_c"]

    if value is None:
        state = "missing"

    elif type(value) not in (int, float):
        state = "invalid"

    elif value >= 28.0:
        state = "alert"

    else:
        state = "normal"

    print(sample["seq"], value, state)
```

실행 조건: Python 3 표준 라이브러리만 사용합니다. 장치·포트·전원을 연결하지 않습니다.

실행 폴더: examples/course

명령: `python -X utf8 code/02_sensor_replay.py`

결과:

```
1 27.5 normal
2 28.0 alert
3 None missing
4 0.0 normal
5 29.0 invalid
```

각 줄의 뜻은 [코드 해설](#)에 있습니다.

작은 수정: 코드의 28.0을 29.0으로 바꾸면 2번은 normal이 됩니다. 문자열 "29.0"은 계속 invalid입니다.

오류 복구: JSON이 깨지면 큰따옴표와 쉼표를 복구합니다. 값 없음과 0을 바꿔 오류를 숨기지 않습니다.

A09 · 직접 해보기

A09 · 모의 센서 파일을 읽기

목적 같은 입력으로 조건 변경 전후를 비교할 수 있어 실물 연결 전 처리 논리를 확인하기 좋습니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A09_input.md](#) · [02_sensor_replay.py](#) · [sensor_samples.json](#)

작업: [A09_worksheet.md](#)

단계

1. 코드 분기 순서를 읽고 출력5개를 예상합니다.
2. 실행 또는 제공 출력과 대조합니다.
3. 경계29.0 수정 결과와 미확인 범위를 씁니다.

예상 결과 입력 근거를 표시하고 예상과 출력에 판단을 남깁니다.

확인 방법 모의 입력으로 확인한 것과 못 한 것은 무엇인가요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 아날로그 신호와 ADC / 센서 기록의 스키마

[이 교사의 다른 예와 코드 해설 열기](#)

4교시 · 경계·흔들림·원래 값 보존

14:00~14:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A10 · 조건의 경계와 결측

4교시 필수 개념 보충 · 경계값과 상태를 기억하는 판단

실무 상황과 원리

가상 온도가 27.9와 28.1 사이를 오가면 경보가 켜졌다 꺼졌다 반복될 수 있습니다. 한 문턱만 사용하는 규칙은 현재 값만 봅니다. 서로 다른 켜짐과 꺼짐 문턱을 사용하는 히스테리시스는 이전 상태도 기억합니다. 실제 장비 제어에 적용하려면 별도의 위험과 정격 검토가 필요하며 오늘은 상태 계산만 합니다.

제작 규칙은 28.0 이상이면 alert=True, 27.5 이하이면 False, 사이이면 이전 상태 유지입니다. 초기 상태 False와 입력 [27.8,28.1,27.9,27.4]를 차례로 추적하면 False,True,True,False입니다. 27.9에서 False로 바뀌지 않는 이유는 중간 구간이기 때문입니다. 28.0과 27.5는 각각 켜짐과 꺼짐에 포함됩니다. 초기 상태가 달라지면 첫 중간 값의 출력도 달라지므로 시작 조건을 기록합니다.

None은 온도가 아니라 측정 누락입니다. 예제는 missing과 no command를 출력하고 이전 내부 상태를 유지합니다. 이는 통신 실패 시 실제 부하를 안전하게 제어하는 정책이 완성되었다는 뜻은 아닙니다. 보존한 상태와 현재 유효한 측정이 있다는 사실을 구분합니다. 완성 추적표에는 순번, 원시값, 이전 상태, 적용한 조건, 새 상태, 측정 유효 여부를 둡니다. elif는 앞 조건이 거짓인 경우에만 다음 조건을 확인하고, continue는 이번 반복의 나머지 처리를 건너뛸니다. 결측 행을 지우거나 이전 온도를 새 측정처럼 복사하면 왜 판단이 유지되었는지 알 수 없습니다.

새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: 초기 False, 새 입력 [28.0,27.7,None,27.5]의 상태와 결측 시 출력 메시지를 순서대로 작성하세요. 중간 27.7이 True인 이유도 설명하세요.

정답과 오답의 이유

True,True,상태 True 유지와 no command,False입니다. 27.7은 두 문턱 사이입니다. None을 영도로 바꾸어 꺼짐 조건에 넣는 답은 결측과 수치를 혼동한 오답입니다. 실제 출력 장치의 안전 정책은 아직 시험하지 않았습니다.

확인 질문과 답

1. 중간 구간은? → 이전 상태를 유지합니다.
2. 초기 상태가 필요한 이유는? → 첫 중간 값의 판단이 달라질 수 있습니다.
3. 결측 출력은 새 온도인가요? → 아닙니다.

조건 경계는 처리 결과가 바뀌는 지점입니다. >=는 기준과 같아도 참입니다. 결측과 잘못된 자료형을 먼저 걸러야 숫자 비교를 정확히 설명할 수 있습니다.

평소 값만 시험하면 같음·이상·없음의 차이를 놓칩니다. 문턱 바로 앞·위·없는 입력을 나란히 둡니다.

입력에서 살펴볼 예

기준28.0 27.9 / 28.0 / 28.1 null / "28.0" / true

27.9 normal 28.0·28.1 alert null missing · 문자/불리언 invalid

구분해야 할 반례

Python에서bool은int의 하위 타입이지만 이 센서 스키마에서는 온도가 아닙니다. 제공 코드는 type을 엄격히 검사합니다.

스스로 설명할 질문: 28.0과문자열"28.0"이 다른 이유를 말할 수 있나요?

활동 연결: [입력](#) · [기록 양식](#)

A10 · 직접 해보기

A10 · 조건의 경계와 결측

목적 평소 값만 시험하면 같음·이상·없음의 차이를 놓칩니다. 문턱 바로 앞·위·없는 입력을 나란히 둡니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A10_input.md](#)

작업: [A10_worksheet.md](#)

단계

1. 여섯 입력의 예상 상태를 씁니다.
2. 숫자·문자·불리언을 분류합니다.
3. 검사 순서와 근거를 설명합니다.

예상 결과 입력 근거를 표시하고 경계 표에 판단을 남깁니다.

확인 방법 28.0과문자열"28.0"이 다른 이유를 말할 수 있나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

A11 · 흔들림과 히스테리시스

히스테리시스는 이전 상태에 따라 현재 판단이 달라지는 특성입니다. 여기서는 켜기28.0,끄기27.5로 기준을 나눠 작은 흔들림에서 반복 전환을 줄입니다.

경계 하나만 쓰면27.9와28.1이 번갈아올 때 상태가 계속 바뀔 수 있습니다. 표시 안정화의 원리를 모의 값으로 살펴봅니다.

입력에서 살펴볼 예

27.8 → 28.1 → 27.9 → 28.2 → 27.4 → null → 28.3 켜기 $\geq$ 28.0 / 끄기 $\leq$ 27.5

꺼짐 → 켜짐 → 켜짐 → 켜짐 → 꺼짐 → 명령 없음 → 켜짐

구분해야 할 반례

이 예의 누락 처리가 모든 장치의 안전 정책은 아닙니다. 실제 제어는 장치 특성에 맞는 별도 실패 안전 정책이 필요합니다.

스스로 설명할 질문: 27.9에서 직전 상태를 왜 기억하나요?

활동 연결: [입력](#) · [기록 양식](#)

03_hysteresis.py 전체

```
values = [27.8, 28.1, 27.9, 28.2, 27.4, None, 28.3]

alert = False

for value in values:
    if value is None:
        print("missing", "no command")
        continue

    if value >= 28.0:
        alert = True

    elif value <= 27.5:
        alert = False

    print(value, alert)
```

실행 조건: Python 3 표준 라이브러리만 사용합니다. 장치·포트·전원을 연결하지 않습니다.

실행 폴더: examples/course

명령: `python -X utf8 code/03_hysteresis.py`

결과:

```
27.8 False
28.1 True
27.9 True
28.2 True
27.4 False
missing no command
28.3 True
```

각 줄의 뜻은 [코드 해설](#)에 있습니다.

작은 수정: 꺼짐 기준 27.5를 27.9로 바꾸면 세 번째 27.9에서 False가 됩니다.

오류 복구: 상태 변수를 반복문 안에서 매번 False로 초기화하면 이력 효과가 사라집니다. 들여쓰기와 초기화 위치를 확인합니다.

A11 · 직접 해보기

A11 · 흔들림과 히스테리시스

목적 경계 하나만 쓰면27.9와28.1이 번갈아올 때 상태가 계속 바뀔 수 있습니다. 표시 안정화의 원리를 모의 값으로 살펴봅니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A11_input.md](#) · [03_hysteresis.py](#)

작업: [A11_worksheet.md](#)

단계

- 1. 상태를 순서대로 예상합니다.
- 2. 완성 코드를 실행하거나 출력과 비교합니다.
- 3. 끄기 기준27.9의 변화를 기록합니다.

예상 결과 입력 근거를 표시하고 상태 추적에 판단을 남깁니다.

확인 방법 27.9에서 직전 상태를 왜 기억하나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 원시값과 판단을 함께 보존

[이 교시의 다른 예와 코드 해설 열기](#)

5교시 · 펌웨어·통신·배선 검토

15:00~15:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A15 · 배선 전 검토표 작성

배선 검토표는 연결 전에 신호·전원·기준·정격·핀 근거를 확인하는 문서입니다. 공사를 시작하기 전 도면을 검토하듯 확인하지 못한 항목은 미정으로 남깁니다.

추측한 핀이나 전압을 실행 계획으로 바꾸지 않습니다. 이번 수업은 종이 연결 계획과 확인 질문까지 작성하며 통전하지 않습니다.

입력에서 살펴볼 예

항목: 보드 모델,핀맵,전원,신호 전압,센서 모델,극성 현재: 모두 장비 정보 미확정

실제 핀 번호는 빈칸으로 남깁니다. 공식 문서의 확인 위치를 적습니다. 통전 결정은 보류합니다.

구분해야 할 반례

목표가 있다고 연결을 추측해서 완성하지 않습니다. 젖은 손·손상 케이블·금속 단락·전원 연결 중 배선 변경은 피하는 기본 원칙을 적용합니다.

스스로 설명할 질문: 미정 표시는 실패가 아니라 어떤 역할을 하나요?

활동 연결: [입력](#) · [기록 양식](#)

A15 · 직접 해보기

A15 · 배선 전 검토표 작성

목적 추측한 핀이나 전압을 실행 계획으로 바꾸지 않습니다. 이번 수업은 종이 연결 계획과 확인 질문까지 작성하며 통전하지 않습니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A15_input.md](#) · [wiring-review.md](#)

작업: [A15_worksheet.md](#)

단계

1. 검토표 각 항목의 필요 이유를 씁니다.
2. 근거 없는 핀 번호와 전압 추정을 지웁니다.
3. 강사가 확인할 순서를 정합니다.

예상 결과 입력 근거를 표시하고 배선 검토표에 판단을 남깁니다.

확인 방법 미정 표시는 실패가 아니라 어떤 역할을 하나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: setup-loop와 직렬 출력 / 시리얼 통신과 파일 재생의 차이

[이 교시의 다른 예와 코드 해설 열기](#)

6교시 · 새 요구·종합 확인·인수인계

16:00~16:50 · 설명15분 / 시연8분 / 직접 수행20분 / 풀이7분

A17 · 종합 모의 검수

종합 검수는 새로운 입력에 같은 규칙을 적용하고 출력·원시값·미확인 항목을 함께 남기는 일입니다. 프로그램의 성공 메시지와 데이터의 의미를 따로 확인합니다.

익숙한 다섯 입력만 외우지 않고 새 순서에서0·경계·미측정·형식 오류를 구분합니다.

입력에서 살펴볼 예

새 입력: 0.0 /27.9 /28.0 /null /"28.1" 기준28.0 source를sensor_new.json으로수정

예상: normal,normal,alert,missing,invalid 센서 실측이 아니라 파일 모의입니다.

구분해야 할 반례

실행이 정상 종료되어도 invalid항목은 남아 있을 수 있습니다. 원문 타입 오류를 검수 결과에 기록합니다.

스스로 설명할 질문: 정상 종료와 모든 측정값 정상의 차이를 설명할 수 있나요?

활동 연결: [입력](#) · [기록 양식](#)

새 입력의 예상과 실제를 나란히 기록

모의 기준은 28.0입니다. N=정상, A=경계 이상 알림, M=미측정, I=잘못된 자료형입니다. 상태는 실제 보드 측정 결과가 아닙니다.

입력 값(다음표 보존)	자료형	예상 상태·이유	실제 상태	확인 방법(직접 실행/제공 출력 관찰)
0.0	_____	_____	_____	_____
27.9	_____	_____	_____	_____

입력 값(따옴표 보존)	자료형	예상 상태·이유	실제 상태	확인 방법(직접 실행/제공 출력 관찰)
28.0	_____	_____	_____	_____
null	_____	_____	_____	_____
"28.1"	_____	_____	_____	_____

- 사용 입력·규칙·출력 위치: _____
- 프로그램 정상 종료 여부: _____
- 정상 종료와 모든 측정값이 정상이라는 말이 다른 이유: _____
- 실제 장비에서 미확인인 모델·전압·핀·연결 자료: _____
- 확인 전 보류할 항목과 확인할 자료: _____

A17 · 직접 해보기

A17 · 종합 모의 검수

목적 익숙한 다섯 입력만 외우지 않고 새 순서에서 0·경계·미측정·형식 오류를 구분합니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A17_input.md](#) · [sensor_new.json](#) · [02_sensor_replay.py](#)

작업: [A17_worksheet.md](#)

단계

1. 새 입력의 상태를 먼저 예상합니다.
2. work복사 코드의 입력 파일명만 바꿔 비교합니다.
3. 원시값·출력·미확인장비 조건을 제출합니다.

예상 결과 입력 근거를 표시하고 종합 예상·결과에 판단을 남깁니다.

확인 방법 정상 종료와 모든 측정값 정상의 차이를 설명할 수 있나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

새 입력의 예상과 실제를 나란히 기록

모의 기준은 28.0입니다. N=정상, A=경계 이상 알림, M=미측정, I=잘못된 자료형입니다. 상태는 실제 보드 측정 결과가 아닙니다.

입력 값(따옴표 보존)	자료형	예상 상태·이유	실제 상태	확인 방법(직접 실행/제공 출력 관찰)
0.0	_____	_____	_____	_____
27.9	_____	_____	_____	_____
28.0	_____	_____	_____	_____
null	_____	_____	_____	_____
"28.1"	_____	_____	_____	_____

- 사용 입력·규칙·출력 위치: _____
- 프로그램 정상 종료 여부: _____
- 정상 종료와 모든 측정값이 정상이라는 말이 다른 이유: _____
- 실제 장비에서 미확인인 모델·전압·핀·연결 자료: _____
- 확인 전 보류할 항목과 확인할 자료: _____

수정 전 예상: _____
확인한 결과와 이유: _____

A18 · 근거를 남겨 다음 팀에 전달

인수인계는 확인한 사실과 아직 확인하지 못한 조건을 다른 사람이 이어갈 수 있게 적는 일입니다. 결과 숫자만이 아니라 입력·코드·설정·판단에 사용한 값과 조건가 필요합니다.

다음 팀이 장비를 연결하기 전에 어떤 시험을 추가해야 하는지 알 수 있게 합니다. 예제로 확인한 것과 장비에서 더 시험할 것을 함께 설명합니다.

입력에서 살펴볼 예

확인한 내용: 파일 판단과 수정·오류경로 추가로 확인할 내용: 실물회로·보드빌드·수신 다음: 모델 확정→문서검토→별도 시험

수치 단위, 0/null, 모의/실물을 구분합니다. 판단에 사용한 값과 조건과 보류 항목을 남깁니다.

구분해야 할 반례

실물 미검증을 숨기고 완료라고 적지 않습니다. 반대로 모의 시험 입력과 결과도 버리지 말고 재사용할 수 있게 남깁니다.

스스로 설명할 질문: 검증했다는 문장을 어떤 근거와 함께 쓸 수 있나요?

활동 연결: [입력](#) · [기록 양식](#)

A18 · 직접 해보기

A18 · 근거를 남겨 다음 팀에 전달

목적 다음 팀이 장비를 연결하기 전에 어떤 시험을 추가해야 하는지 알 수 있게 합니다. 예제로 확인한 것과 장비에서 더 시험할 것을 함께 설명합니다.

준비 해당 data 입력 카드와 work 기록 양식을 엽니다. 실제 보드·전선·전원은 연결하지 않습니다. 계산기 또는 제공 출력 비교로 참여할 수 있습니다.

입력: [A18_input.md](#)

작업: [A18_worksheet.md](#)

단계

1. 전압·전류·저항을 한 문장씩 설명합니다.
2. 종합 결과에 입력 파일과 규칙을 연결합니다.
3. 장비 확정 후 필요한 시험 세 개를 적습니다.

예상 결과 입력 근거를 표시하고 핵심 설명에 판단을 남깁니다.

확인 방법 검증했다는 문장을 어떤 근거와 함께 쓸 수 있나요?

오류 해결 단위·입력 값·적용 조건을 원문과 대조합니다. 실물 연결로 오류를 확인하지 않습니다.

힌트: 단어만 쓰지 말고 입력 카드의 값 하나를 가리키며 이유를 말하게 합니다.

선택 확장: 조건 한 가지를 바꿔 예상과 한계를 함께 적습니다.

수정 전 예상: _____

확인한 결과와 이유: _____

함께 보는 예와 더 해보기: 새 센서 요구를 시험으로 바꾸기

[이 교시의 다른 예와 코드 해설 열기](#)

오늘 배운 내용 확인

확인할 것	배점	충족 기준
배운 내용 설명	20점	사용한 개념과 선택한 이유를 예로 설명한다.
결과물 완성	40점	오늘 필수 활동의 파일·표·기획 내용을 빠짐없이 완성한다.
결과 확인	30점	예상과 결과를 비교하고 다른 입력 한 가지로 다시 확인한다.

저장과 전달	10점	파일 이름과 실행·읽는 방법을 함께 적어 다른 사람이 확인할 수 있다.
--------	-----	---

각 항목: 모두 충족하면 배점 전부, 일부 충족하면 배점의 절반, 근거가 없으면 0점. 오류를 발견하고 고친 기록도 확인 근거로 사용합니다.

남길 결과: 종합 예상·결과 / 아직 확인하지 못한 조건 / 핵심 설명 / 다음 팀에 전달할 내용

세 가지 확인 질문

1. 센서 기록의 숫자 0과 null은 어떻게 다른가요? 결과 화면에서는 어떻게 표시할까요?
내 설명: _____
2. 오늘 필수 활동에서 값을 한 곳 바꾼 예를 고르세요. 바꾸기 전 예상과 확인한 결과를 설명해 보세요.
내 설명: _____
3. 마지막 결과물의 파일 이름과 읽거나 실행하는 방법을 적어 보세요. 다른 사람이 확인할 수 있도록 한 문장씩 안내합니다.
내 설명: _____

더 해보기 · 추가 확인 문제

설명으로 확인하는 질문

1. 저항 양단에 3.3V가 걸리고 저항이 660Ω이라면 전류는 얼마인가요? A와 mA 단위를 구별해 설명하세요.
2. ADC 원시값 하나만 보고 온도라고 말할 수 있나요? 해석에 필요한 추가 정보를 적으세요.
3. null과 0.0을 같은 값으로 처리하면 어떤 정보를 잃게 되나요? 센서 측정 상황으로 설명하세요.
4. 파일을 사용한 모의 예제가 통과한 뒤, 실제 보드를 연결하기 전에 더 확인할 사항 세 가지는 무엇인가요?

새 상황으로 옮겨 보는 다른 상황에 적용하는 문제 · 복습·도전

교시당 한 문제이며 기존 필수 평가 후 필요에 따라 선택합니다. 독립 작성 또는 판단을 먼저 기록한 뒤 해설을 확인합니다. 설치 실패나 장비 부재는 개념 오답과 구분합니다.